

XAI Project

Heart Failure Clinical Records

Confronto tra modelli interpretabili basati su regole
e modelli ensemble non interpretabili

Indice

1. Introduzione del problema e dataset

1.1	Contesto e motivazione	2
1.2	Il Dataset	3
1.3	EDA — Distribuzioni per classe	4
1.4	EDA — Matrice di correlazione	5

2. Architettura e Implementazione

2.1	Struttura del Progetto	6
2.2	Pipeline di Elaborazione dei Dati	7
2.3	Modelli Interpretabili Basati su Regole	9
2.4	Modelli Ensemble Non Interpretabili	10
2.5	Configurazione e Iperparametri	11

3. Risultati

3.1	Metriche Comparative	12
3.2	Curve ROC	13
3.3	Matrici di Confusione	14
3.4	Interpretabilità — Regole RuleFit	15
3.5	Regole Complete: GreedyRuleList e BayesianRuleList	16
3.6	Interpretabilità — Feature Importance RF	17
3.7	Cross-Validation 10-fold	18

4. Conclusioni

4.1	Confronto, analisi clinica e limiti	19
-----	-------------------------------------	----

Contesto e motivazione

Il problema

L'insufficienza cardiaca è una condizione clinica grave in cui il cuore non riesce a pompare sangue in modo sufficiente. Identificare i fattori di rischio associati al decesso è fondamentale per supportare le decisioni cliniche e migliorare la prognosi dei pazienti.

Obiettivo

Il progetto confronta due famiglie di modelli di machine learning per la predizione della mortalità: modelli interpretabili basati su regole (rule-based) e modelli non interpretabili basati su ensemble di alberi. L'obiettivo è valutare il trade-off tra performance predittiva e interpretabilità in un contesto clinico ad alta criticità.

Explainable AI in ambito clinico

In medicina, un modello non spiegabile — per quanto accurato — non è direttamente utilizzabile come strumento di supporto decisionale: il medico deve poter comprendere e validare le motivazioni di ogni predizione. I modelli intrinsecamente interpretabili soddisfano questo requisito senza richiedere metodi post-hoc aggiuntivi (es. SHAP, LIME), che possono introdurre approssimazioni e spiegazioni fuorvianti.

Struttura del report

Il Capitolo 1 introduce il problema clinico, descrive il dataset e presenta l'analisi esplorativa. Il Capitolo 2 illustra l'architettura del progetto, la pipeline di elaborazione dei dati, i modelli utilizzati e la loro configurazione. Il Capitolo 3 presenta i risultati e il confronto delle performance. Il Capitolo 4 raccoglie le conclusioni, l'analisi clinica dei risultati e i limiti del lavoro.

Il Dataset

Descrizione

Il dataset Heart Failure Clinical Records (UCI, id=519) raccoglie cartelle cliniche di 299 pazienti con insufficienza cardiaca seguiti in follow-up. La variabile target DEATH_EVENT indica se il paziente è deceduto durante il periodo di osservazione.

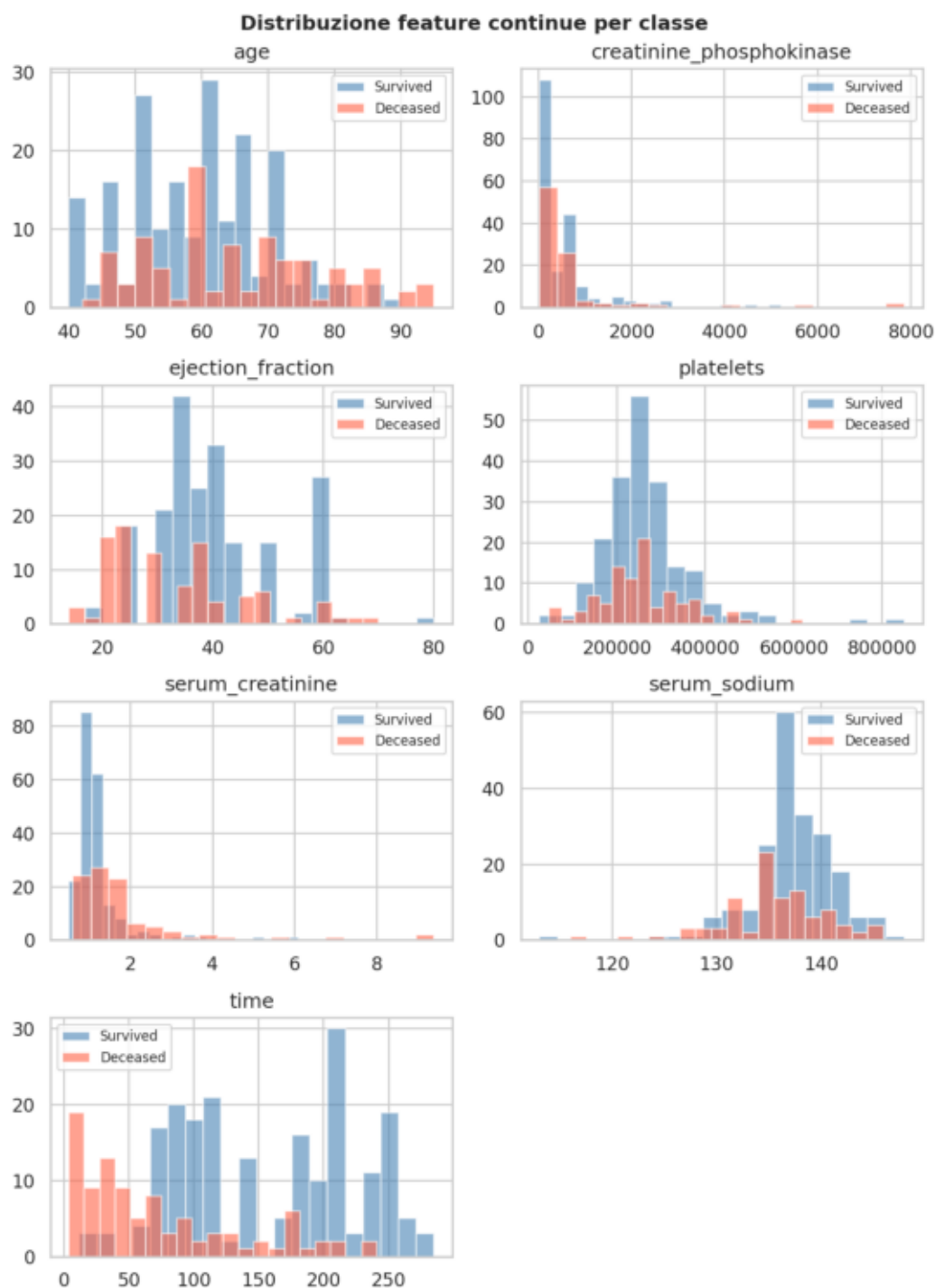
Campioni: 299 | Feature: 12 | Decessi: 96 (32.1%) | Sopravvissuti: 203 (67.9%)

Gestione delle feature: binarie vs continue

Le 12 feature si dividono in 5 binarie (anaemia, diabetes, high_blood_pressure, sex, smoking — valori 0/1) e 7 continue (age, creatinine_phosphokinase, ejection_fraction, platelets, serum_creatinine, serum_sodium, time). I modelli ensemble e la maggior parte dei rule-based le ricevono direttamente. RuleFit richiede standardizzazione (StandardScaler), mentre BayesianRuleList richiede feature esclusivamente binarie: le variabili continue vengono quindi discretizzate in bin tramite KBinsDiscretizer con one-hot encoding prima del fitting.

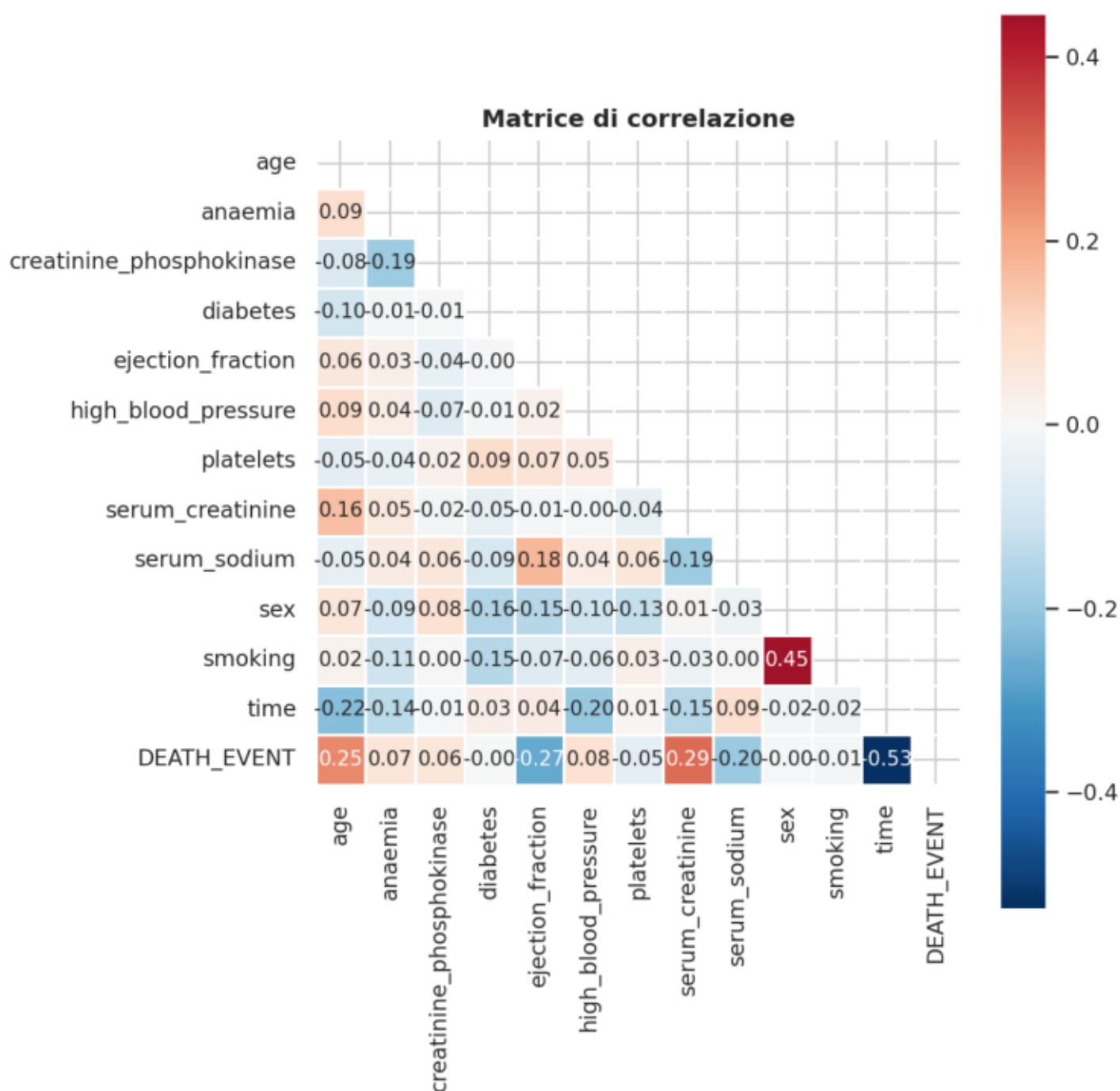
Feature	Tipo	Descrizione
age	continua	Età del paziente (anni)
anaemia	binaria	Presenza di anemia (0/1)
creatinine_phosphokinase	continua	Livello CPK nel sangue (mcg/L)
diabetes	binaria	Presenza di diabete (0/1)
ejection_fraction	continua	Percentuale di sangue pompata dal ventricolo sinistro (%)
high_blood_pressure	binaria	Presenza di ipertensione (0/1)
platelets	continua	Conta piastrinica (kiloplatelets/mL)
serum_creatinine	continua	Livello di creatinina serica (mg/dL)
serum_sodium	continua	Livello di sodio sierico (mEq/L)
sex	binaria	Sesso biologico (0=F, 1=M)
smoking	binaria	Fumatore (0/1)
time	continua	Durata del follow-up (giorni)

EDA — Distribuzioni per classe



Ogni istogramma sovrappone la distribuzione di una feature per sopravvissuti (blu) e deceduti (rosso): più le due curve sono separate, maggiore è il potere predittivo di quella variabile. `time` è il discriminatore più potente — un follow-up breve corrisponde quasi sempre a decesso, segno di rapido deterioramento clinico. `serum_creatinine` alta indica compromissione renale, spesso associata a esiti peggiori; `ejection_fraction` bassa segnala che il ventricolo sinistro pompa una quota insufficiente di sangue. `platelets` e `creatinine_phosphokinase` mostrano distribuzioni largamente sovrapposte: contributo predittivo individuale marginale.

EDA — Matrice di correlazione



Ogni cella mostra la correlazione di Pearson tra due variabili: vale +1 se quando una sale anche l'altra sale sempre, -1 se quando una sale l'altra scende sempre, 0 se non c'è alcuna relazione lineare tra le due. Valori vicini a ± 0.5 o oltre indicano una relazione abbastanza chiara; vicini a 0 indicano variabili sostanzialmente indipendenti. time ha la correlazione più forte con DEATH_EVENT (-0.53): pazienti con follow-up lungo tendono a sopravvivere. serum_creatinine (+0.29) ed ejection_fraction (-0.27) sono i segnali biochimici più informativi. Le feature binarie mostrano correlazioni deboli, sotto ± 0.15 .

Struttura del Progetto

Organizzazione del codice

Il progetto è organizzato in una cartella `src/` con sei moduli distinti, ciascuno con una responsabilità precisa. Questa separazione permette di modificare una componente senza toccare le altre e rende il codice più leggibile e testabile.

Modulo	Responsabilità
<code>config.py</code>	Centralizza iperparametri, path e costanti — unico punto di modifica
<code>data_loader.py</code>	Download UCI, cache CSV, split stratificato, tre varianti di preprocessing
<code>models.py</code>	Factory function per ciascuno degli 8 modelli — nessuna logica di training
<code>evaluate.py</code>	Calcolo metriche, generazione di tutti i grafici, cross-validation
<code>main.py</code>	Orchestrazione dell'intera pipeline end-to-end, in ordine fisso
<code>report.py</code>	Lettura degli artefatti salvati e generazione del PDF report

Output della pipeline

Ogni esecuzione di `main.py` produce artefatti salvati su disco: i dati grezzi vengono cachati in `data/` (`X.csv`, `y.csv`) per evitare di riscargarli ogni volta. Le metriche vengono salvate in `results/metrics_summary.csv`. I grafici vengono salvati in `results/plots/` come PNG ad alta risoluzione (150 DPI). Il report finale viene scritto in `results/report.pdf`. Questa separazione tra esecuzione e report permette di rigenerare il PDF senza rieseguire l'intera pipeline.

Dipendenze principali

`scikit-learn` gestisce preprocessing, metriche e cross-validation. `imodels` fornisce le implementazioni dei modelli rule-based (`RuleFit`, `GreedyRuleList`, `BayesianRuleList`, `FIGS`, `SkopeRules`). `xgboost` implementa XGBoost. `ucimlrepo` scarica il dataset direttamente dal repository UCI. `matplotlib` e `seaborn` gestiscono la generazione di tutti i grafici e del report PDF.

Pipeline di Elaborazione dei Dati

Split stratificato

Il dataset viene diviso in training (80%, 239 campioni) e test (20%, 60 campioni) con `train_test_split(stratify=y)`: la stratificazione garantisce che entrambe le parti mantengano la stessa proporzione di deceduti (~32%) presente nel dataset originale. Senza stratificazione, con un dataset così piccolo, uno split casuale potrebbe produrre un test set con proporzioni molto diverse, rendendo le metriche poco affidabili. Il seed fisso (`RANDOM_STATE=42`) garantisce la riproducibilità.

Tre varianti di preprocessing

I modelli richiedono rappresentazioni diverse delle stesse feature, quindi il preprocessing produce tre versioni del training e test set in parallelo. La versione raw è il DataFrame originale con le 12 feature così come sono: usata da `GreedyRuleList`, `FIGS`, `SkopeRules` e dai modelli ensemble (come array NumPy). La versione scalata applica `StandardScaler`, che porta ogni feature continua a media 0 e varianza 1: necessaria per `RuleFit`, che usa la regressione Lasso internamente e quindi è sensibile alla scala delle variabili. La versione discretizzata usa `KBinsDiscretizer(n_bins=4, encode='onehot-dense', strategy='quantile')`: ogni feature continua viene suddivisa in 4 intervalli di uguale frequenza, ciascuno codificato come colonna binaria. Il risultato è un array di circa 33 colonne binarie (le feature continue producono 4 bin ciascuna, le 5 binarie ne producono 1), obbligatorio per `BayesianRuleList` che per design accetta solo valori strettamente 0/1.

Pipeline di Elaborazione dei Dati (continua)

PreparedData — struttura dati condivisa

Il risultato del preprocessing è incapsulato in un dataclass `PreparedData` che raccoglie tutte le versioni in un unico oggetto: `X_train/X_test (raw)`, `X_train_scaled/X_test_scaled`, `X_train_disc/X_test_disc`, `y_train/y_test`, la lista dei nomi delle feature e `scale_pos_weight` per XGBoost. `main.py` accede ai campi tramite `data.X_train_scaled`, `data.y_test` ecc., senza dover gestire decine di variabili separate. Lo scaler e il discretizzatore sono fittati solo sul training set e applicati al test set con `transform()`, evitando data leakage.

Sbilanciamento delle classi

Il dataset contiene 203 sopravvissuti e 96 deceduti (~32% positivi). Questo sbilanciamento è gestito in modo diverso per ciascun modello: Random Forest usa `class_weight='balanced'` che pesa ogni campione inversamente alla frequenza della sua classe. XGBoost usa `scale_pos_weight` calcolato automaticamente come rapporto tra negativi e positivi nel training set. Gli altri modelli non hanno meccanismi nativi e trattano le classi con peso uguale.

Modelli Interpretabili Basati su Regole

I modelli basati su regole producono predizioni nella forma di liste di condizioni if-then direttamente leggibili. Ogni regola può essere validata da un esperto di dominio senza strumenti aggiuntivi.

RuleFit

Estrae regole da un ensemble di alberi, poi allena una regressione lineare penalizzata (Lasso) sui valori delle regole. Il coefficiente di ogni regola indica il suo peso nella predizione finale.

Richiede feature standardizzate.

GreedyRuleList

Costruisce una lista ordinata di regole if-then in modo greedy, massimizzando ad ogni passo la riduzione dell'impurità. La lista si legge dall'alto verso il basso: si applica la prima regola che si attiva sul campione.

BayesianRuleList (BRL)

Apprende una lista di regole tramite inferenza Bayesiana (MCMC). Richiede feature binarie: le variabili continue vengono discretizzate in bin (one-hot encoding) prima del fitting. Produce liste compatte e altamente leggibili.

FIGS — Fast Interpretable Greedy-Tree Sums

Il modello è una somma di piccoli alberi decisionali, ciascuno appreso in modo greedy sui residui del passo precedente. Combina interpretabilità strutturale con una capacità espressiva maggiore rispetto a un singolo albero.

SkopeRules

Genera regole ad alta precisione e recall tramite sub-campionamenti ripetuti di ensemble di alberi, poi de-duplica le regole simili. Ogni regola è corredata da statistiche di precisione, recall e support sul training set.

Modelli Ensemble Non Interpretabili

I modelli ensemble combinano centinaia di alberi decisionali per ottenere performance superiori, ma il processo decisionale risulta opaco (black box). Sono usati come baseline di riferimento per valutare il gap di performance rispetto ai modelli interpretabili.

Random Forest

Allena N alberi decisionali su sotto-campioni casuali del dataset (bagging) e su sottoinsiemi casuali delle feature (feature bagging). La predizione finale è la media delle probabilità dei singoli alberi. Riduce la varianza rispetto al singolo albero. Configurato con `class_weight='balanced'` per gestire lo sbilanciamento.

Gradient Boosting (sklearn)

Allena alberi sequenzialmente: ogni albero corregge gli errori del precedente minimizzando il gradiente della loss (boosting). Riduce il bias rispetto al bagging. Parametri: 200 stimatori, learning rate 0.05, profondità massima 4, subsample 0.8.

XGBoost

Implementazione ottimizzata del gradient boosting con regolarizzazione L1/L2, gestione nativa dei valori mancanti e parallelismo. Parametro `scale_pos_weight` impostato automaticamente per bilanciare le classi in base alla distribuzione del training set.

Configurazione e Iperparametri

Tutti i parametri del progetto sono centralizzati in `config.py`, importato da ogni modulo. Questo garantisce che una singola modifica si propaghi all'intera pipeline senza rischio di disallineamenti. Di seguito la motivazione di ogni scelta.

Parametri generali

Parametro	Valore	Motivazione
RANDOM_STATE	42	Seed fisso per riproducibilità di split e modelli stocastici
TEST_SIZE	0.2	20% test (60 campioni); bilancia valutazione e training
CV_FOLDS	10	10-fold CV: stima robusta su dataset piccoli
BINARY_FEATURES	(5)	Feature binarie escluse da StandardScaler

Modelli rule-based

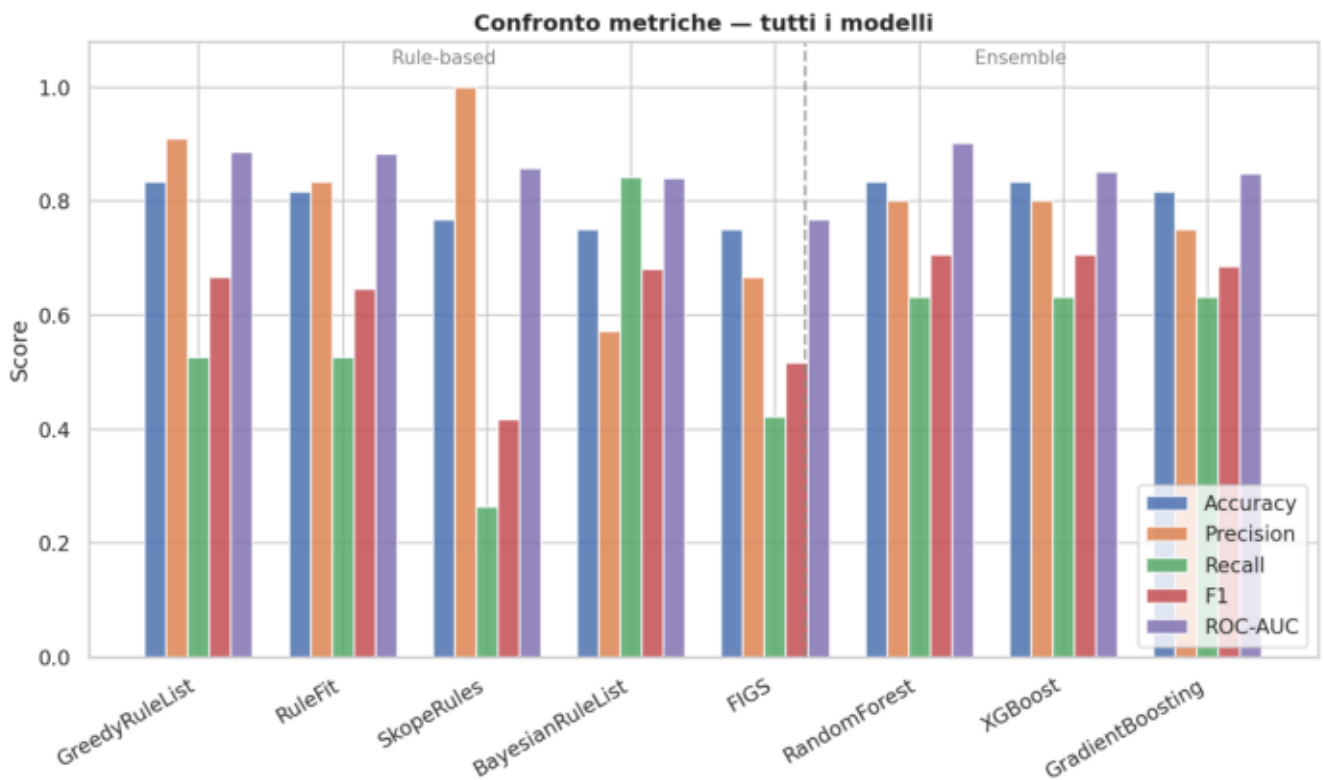
Parametro	Valore	Motivazione
RULEFIT_MAX_RULES	30	Limita le regole candidate; troppe degenerano in black-box
GRL_MAX_DEPTH	6	Profondità massima della lista greedy (default libreria)
BRL_MAX_ITER	20 000	Iterazioni MCMC sufficienti per la convergenza
BRL_MAX_CARDINALITY	2	Max 2 condizioni per regola: mantiene leggibilità
FIGS_MAX_RULES	10	Alberi nella somma FIGS; oltre 10 perde interpretabilità
SKOPE_N_ESTIMATORS	100	Pool ampio di alberi base per diversità delle regole
SKOPE_PRECISION_MIN	0.5	Soglia minima di precisione per accettare una regola
SKOPE_RECALL_MIN	0.4	Copertura minima dei positivi nel training per regola

Modelli ensemble

Parametro	Valore	Motivazione
RF_N_ESTIMATORS	200	Abbastanza alberi da stabilizzare le stime RF
GB_N_ESTIMATORS	200	Iterazioni di boosting sufficienti senza overfitting
GB_LEARNING_RATE	0.05	Passi piccoli nel gradiente: riduce il rischio di overfitting
GB_MAX_DEPTH	4	Alberi profondi limitati; il boosting compensa il bias
XGB_N_ESTIMATORS	200	Allineato a GB per confronto diretto
XGB_LEARNING_RATE	0.05	Come GB; XGBoost aggiunge anche regolarizzazione L1/L2
XGB_MAX_DEPTH	4	Come GB: limita i learner deboli nel boosting sequenziale

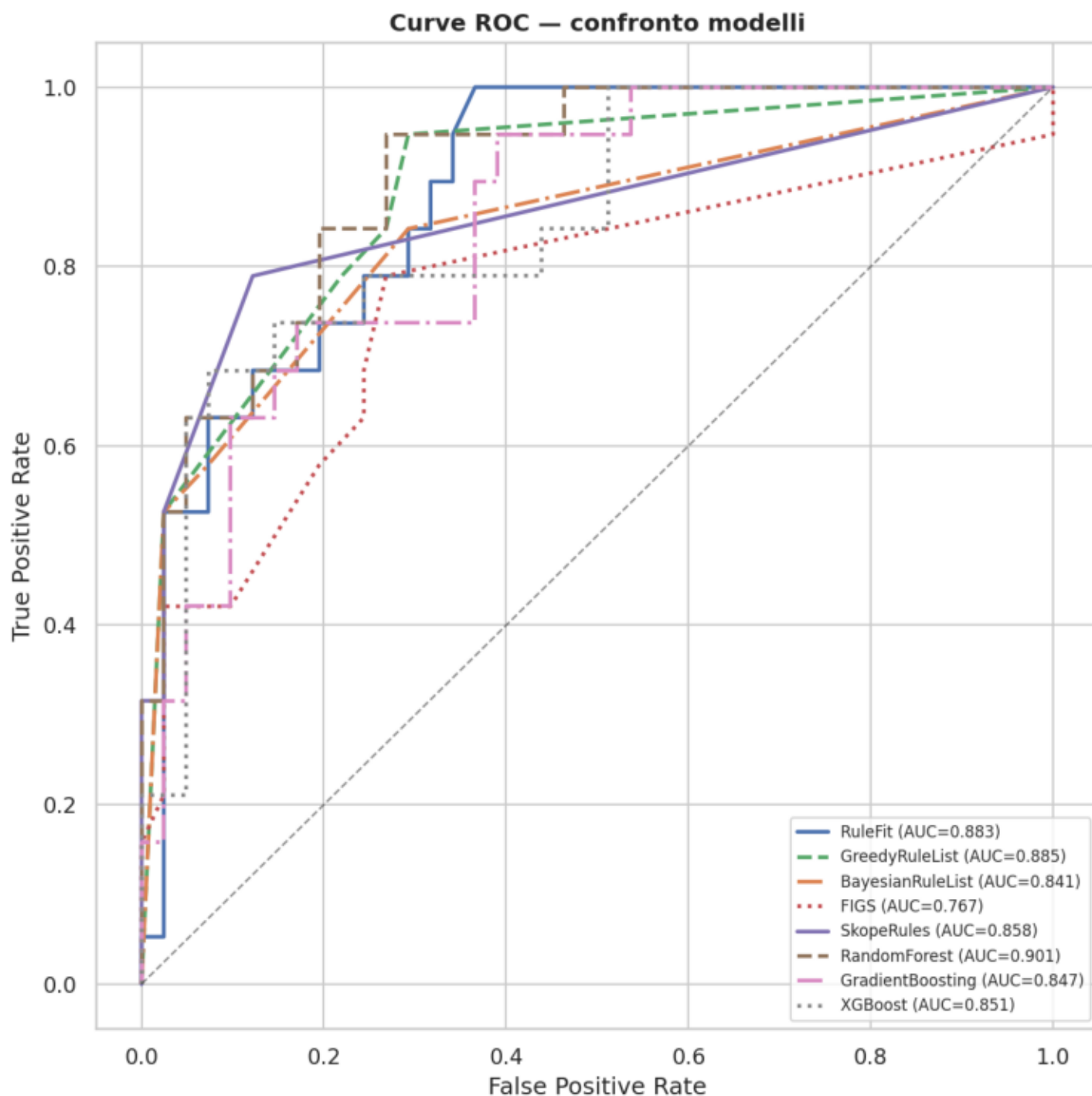
Metriche Comparative

Modello	Type	Accuracy	Precision	Recall	F1	ROC-AUC
RandomForest	Ensemble	0.8333	0.8	0.6316	0.7059	0.9012
GreedyRuleList	Rule-based	0.8333	0.9091	0.5263	0.6667	0.8851
RuleFit	Rule-based	0.8167	0.8333	0.5263	0.6452	0.8825
SkopeRules	Rule-based	0.7667	1.0	0.2632	0.4167	0.8575
XGBoost	Ensemble	0.8333	0.8	0.6316	0.7059	0.8511
GradientBoosting	Ensemble	0.8167	0.75	0.6316	0.6857	0.8472
BayesianRuleList	Rule-based	0.75	0.5714	0.8421	0.6809	0.8408
FIGS	Rule-based	0.75	0.6667	0.4211	0.5161	0.767



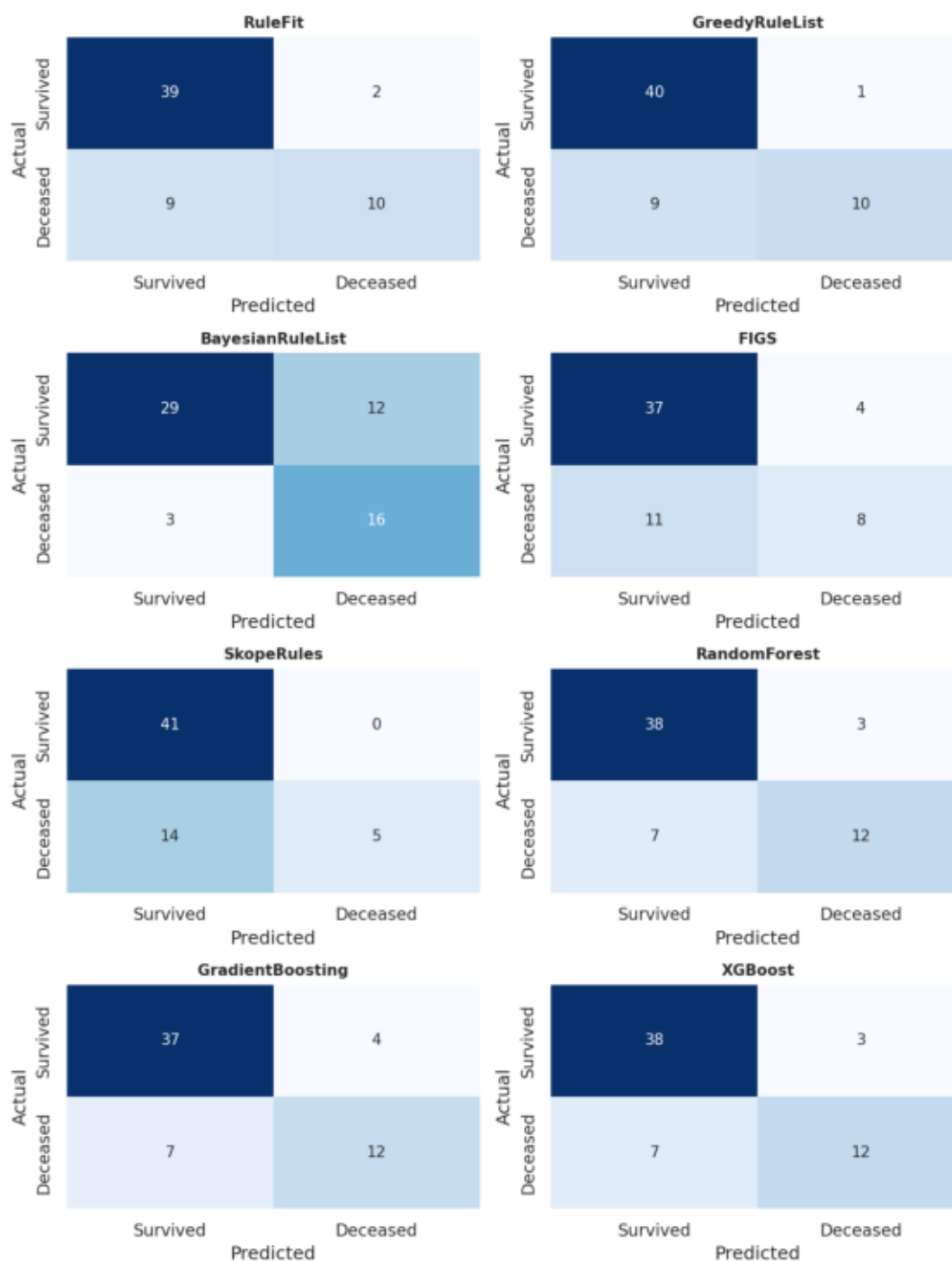
Ogni gruppo di barre rappresenta un modello; i cinque colori corrispondono alle cinque metriche. Accuracy e F1 riassumono la qualità complessiva, ma su classi sbilanciate possono essere fuorvianti. Precision misura quante predizioni 'deceduto' sono corrette; Recall quante morti reali vengono identificate — in clinica un Recall basso (morte non rilevata) è più pericoloso di una Precision bassa (falso allarme). ROC-AUC è la metrica più robusta: misura la capacità di separare le classi indipendentemente dalla soglia. GreedyRuleList è il miglior rule-based per F1 e Accuracy. SkopeRules ottiene ROC-AUC 0.858 con Precision=1.0 e Recall=0.263: identifica pochi deceduti ma senza mai

Curve ROC



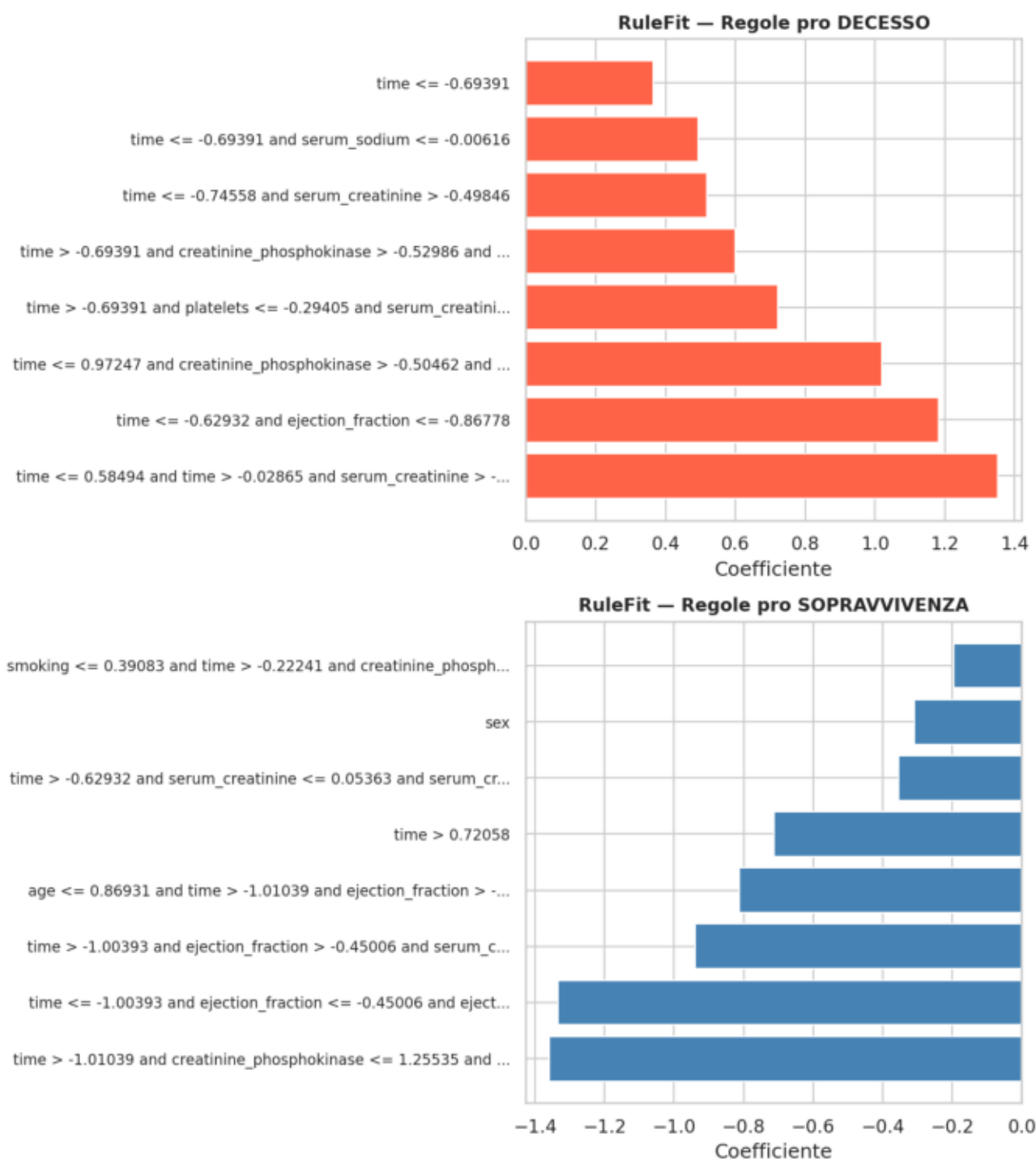
La curva ROC traccia il True Positive Rate (Recall, asse y) contro il False Positive Rate (1 - Specificità, asse x) al variare della soglia di classificazione. Un modello perfetto punta all'angolo in alto a sinistra (TPR=1, FPR=0); uno casuale segue la diagonale tratteggiata (AUC=0.50). L'area sotto la curva (AUC) riassume la performance in un unico numero indipendente dalla soglia: valori sopra 0.80 sono considerati buoni in ambito clinico. Random Forest raggiunge AUC 0.901, il più alto. SkopeRules e GreedyRuleList seguono (~0.89), risultato notevole per modelli intrinsecamente interpretabili. FIGS è il più debole (~0.77), penalizzato dalla struttura rigida.

Matrici di Confusione



Ogni cella indica: riga = classe reale, colonna = classe predetta. La diagonale principale (celle più scure) raccoglie le predizioni corrette; gli elementi fuori diagonale sono gli errori. In clinica i due tipi di errore hanno costo asimmetrico: un falso negativo (morte non rilevata) è più grave di un falso positivo (falso allarme). **BayesianRuleList** massimizza il Recall sui deceduti — pochi falsi negativi — a scapito di più falsi positivi. Gli ensemble (**RandomForest**, **XGBoost**) bilanciano meglio i due errori. **SkopeRules** ha Precision=1.0 ma Recall=0.263: identifica solo 5 deceduti su 19, senza mai produrre falsi positivi — comportamento by design delle sue soglie di accettazione.

Interpretabilità — Regole RuleFit



RuleFit assegna a ogni regola estratta un coefficiente tramite regressione Lasso: coefficiente positivo (barre rosse, grafico superiore) significa che quella condizione aumenta la probabilità di decesso; negativo (blu, inferiore) la diminuisce. Le regole più influenti nel predire il decesso coinvolgono time basso e serum_creatinine alta, confermando i pattern dell'EDA. Le regole pro-sopravvivenza implicano ejection_fraction più alta e follow-up lungo. La penalizzazione Lasso azzerava i coefficienti delle regole ridondanti, mantenendo solo quelle con effetto reale sulla predizione.

Regole Complete: GreedyRuleList e BayesianRuleList

GreedyRuleList

Lista ordinata di regole IF/THEN applicate in sequenza: il primo paziente che soddisfa una condizione viene classificato immediatamente. P(decesso) indica la probabilità stimata di decesso per i campioni che soddisfano la regola; Campioni indica quanti pazienti del training set vengono classificati da quella regola.

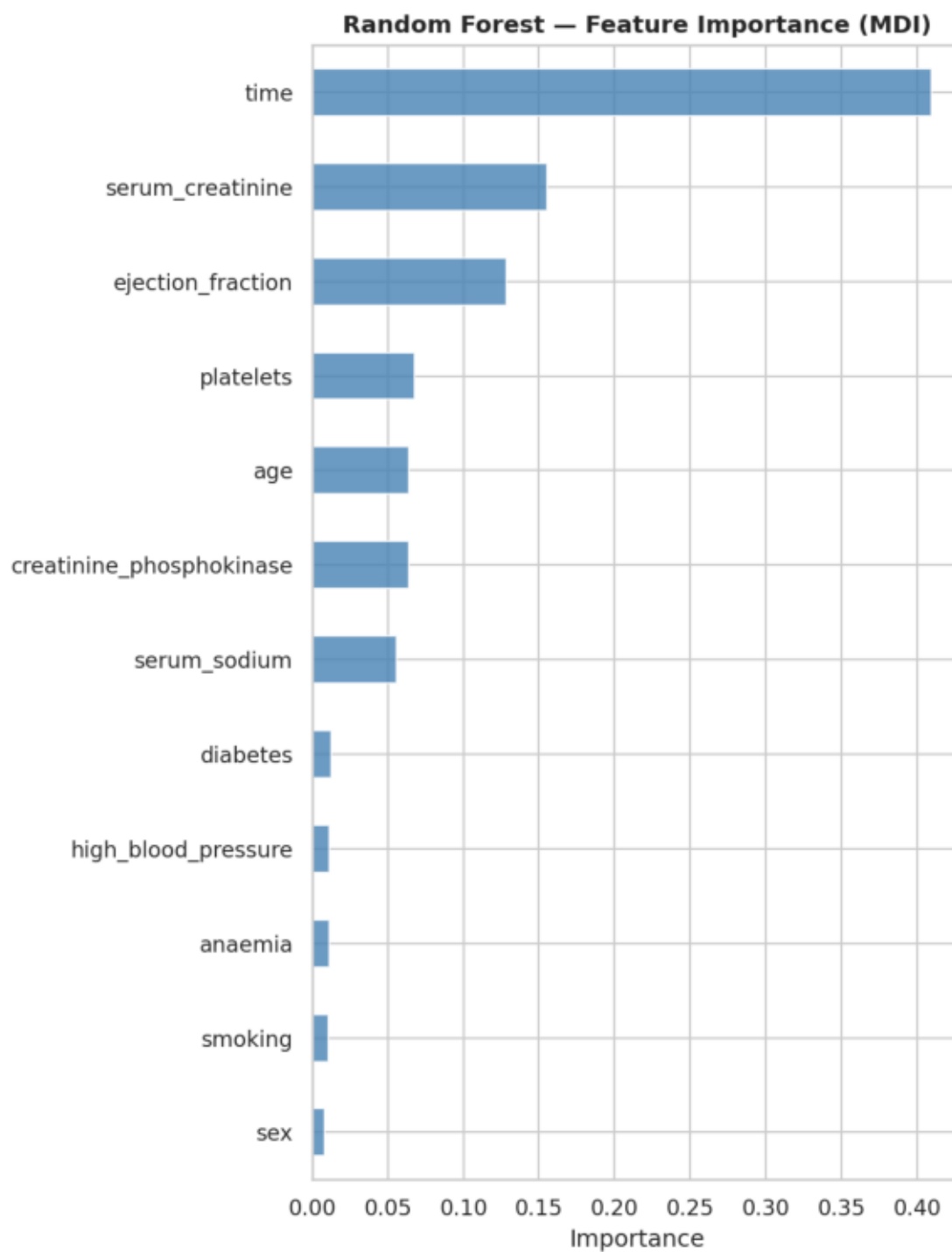
Condizione	P(decesso)	Campioni
time ≤ 73.500	81.5%	65
serum_creatinine > 1.450	46.9%	32
ejection_fraction ≤ 27.500	29.4%	17
creatinine_phosphokinase > 2307.500	25.0%	8
serum_creatinine ≤ 0.650	50.0%	2
age > 79.000	25.0%	4
DEFAULT (nessuna regola attivata)	0.0%	111

BayesianRuleList

Lista IF/ELSE appresa tramite inferenza bayesiana (MCMC) su feature discretizzate in bin. Ogni condizione 'feature ∈ (a, b]' indica che la feature cade in quell'intervallo. P(decesso) è la probabilità posteriore con intervallo di credibilità al 95%.

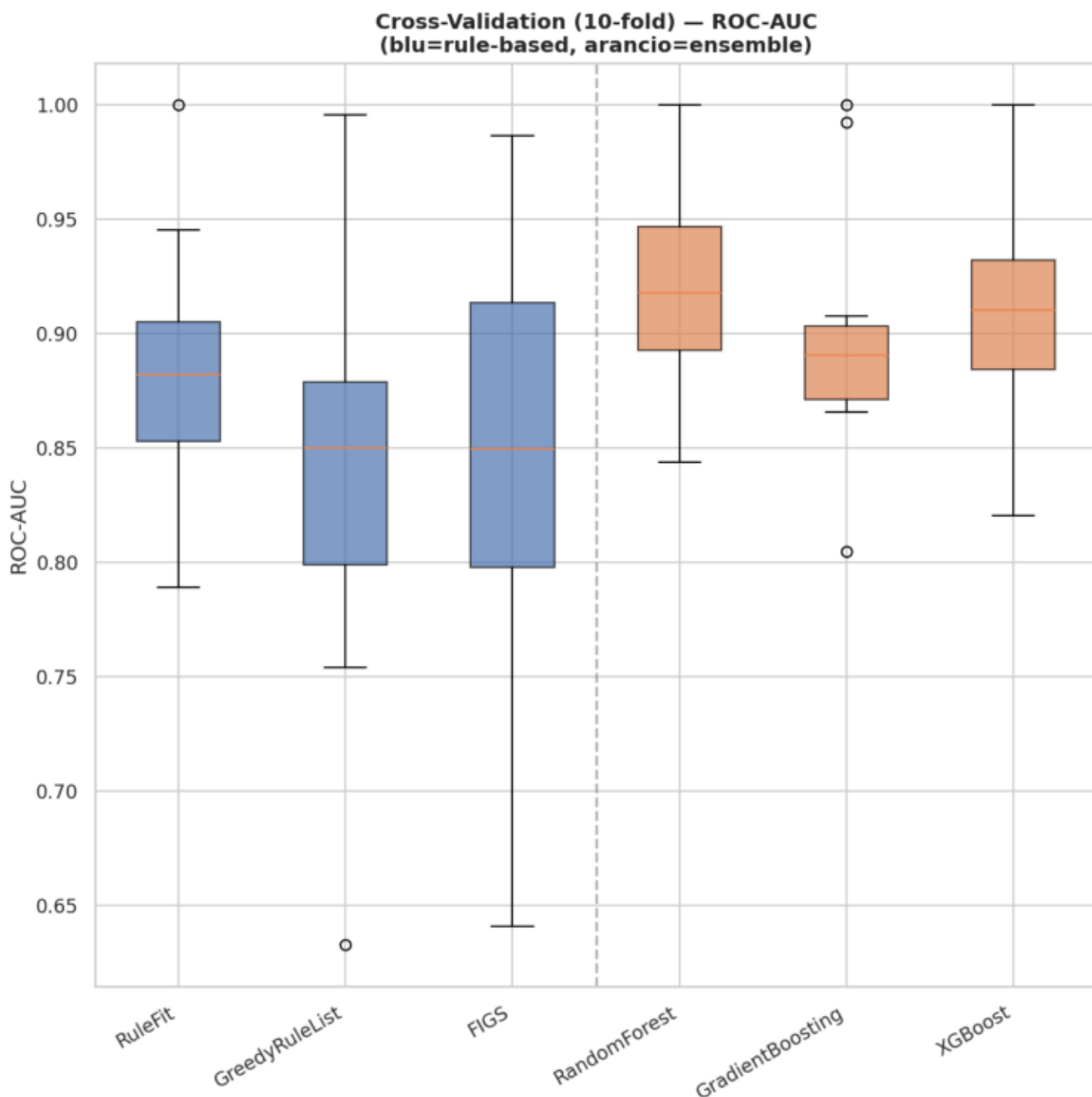
Condizione	P(decesso)	IC 95%
time ∈ (4.00, 69.50]	82.3%	71.9%–90.6%
ejection_fraction ∈ (15.00, 30.00]	47.2%	31.4%–63.4%
serum_creatinine ∈ (1.40, 9.40]	28.0%	12.6%–46.7%
DEFAULT (nessuna regola attivata)	4.8%	1.8%–9.2%

Interpretabilità — Feature Importance RF



L'importanza MDI (Mean Decrease in Impurity) misura di quanto ogni feature riduce l'impurità media nei nodi degli alberi del Random Forest: valori più alti indicano feature più utilizzate e discriminative. `time` domina nettamente, seguita da `serum_creatinine` ed `ejection_fraction` — le stesse tre variabili che emergono dalle regole di `RuleFit` e `GreedyRuleList`: una validazione incrociata tra approcci molto diversi. Le feature binarie (`anaemia`, `diabetes`, `sex`, `smoking`) hanno importanza marginale, coerente con le basse correlazioni dell'EDA.

Cross-Validation 10-fold



Il boxplot mostra la distribuzione del ROC-AUC su 10 fold stratificati: mediana, IQR (bordi del box) e range (baffi). La stratificazione garantisce che ogni fold mantenga la stessa proporzione di deceduti (~32%) del dataset completo, rendendo la stima più affidabile. Random Forest ha la mediana più alta e IQR contenuto: performance stabile e consistente tra i fold. RuleFit è il miglior rule-based in CV. FIGS mostra alta varianza — instabile su dataset piccoli. GreedyRuleList ottiene 0.843 ± 0.096 tramite un custom scorer che aggira un'incompatibilità con l'API sklearn.

Conclusioni

Confronto tra approcci

I modelli ensemble raggiungono ROC-AUC superiore (Random Forest 0.901, XGBoost 0.851) ma il divario con i rule-based non è netto: GreedyRuleList (0.885) e RuleFit (0.883) li superano entrambi. Su soli 299 campioni la semplicità strutturale dei modelli a regole riduce il rischio di overfitting, risultando competitiva. GreedyRuleList offre il miglior F1 rule-based (0.667) con una lista ordinata di condizioni leggibile da un clinico. RuleFit assegna coefficienti espliciti a ogni regola, rendendo visibile il peso relativo di ciascuna. BayesianRuleList privilegia il recall (0.842): sovrastima i decessi, il che in medicina è spesso preferibile a sottostimarli. FIGS ha AUC più basso (0.767) ma struttura additiva particolarmente trasparente. SkopeRules raggiunge Precision=1.0 con Recall=0.263: non produce mai falsi positivi, ma identifica solo 5 deceduti su 19 — utile quando ogni allarme deve corrispondere a un caso realmente critico.

Precision vs Recall in ambito clinico

Il trade-off tra precisione e recall non è neutro in contesti medici. Alta recall minimizza i falsi negativi (morti non rilevate), che in clinica sono più pericolosi dei falsi positivi (falsi allarmi). Alta precisione garantisce invece che ogni segnalazione sia affidabile, preferibile quando si pianificano interventi invasivi. La scelta del modello dipende quindi dal contesto d'uso, e la trasparenza dei modelli rule-based è la condizione necessaria per fare questa scelta in modo consapevole.

Migliori modelli per categoria:

Categoria	Modello	ROC-AUC	F1
Rule-based	GreedyRuleList	0.8851	0.6667
Ensemble	RandomForest	0.9012	0.7059

Conclusioni (continua)

Validazione qualitativa delle regole

Le feature più ricorrenti — time (follow-up), serum_creatinine ed ejection_fraction — sono coerenti con la letteratura clinica sull'insufficienza cardiaca: follow-up breve segnala una fase acuta critica, creatinina alta indica insufficienza renale spesso letale in questo contesto, ejection_fraction bassa riflette ridotta capacità di pompa. Il fatto che i modelli apprendano autonomamente queste relazioni costituisce una validazione qualitativa: catturano pattern fisiopatologici reali, non solo correlazioni statistiche.

Stabilità in cross-validation

Random Forest (0.918 ± 0.044) e XGBoost (0.910 ± 0.049) sono i modelli più stabili. RuleFit raggiunge 0.884 ± 0.058 — il miglior risultato tra i rule-based in CV e competitivo con gli ensemble. GreedyRuleList ottiene 0.843 ± 0.096 , con varianza più alta ma attesa su fold di circa 24 campioni. FIGS ha la varianza maggiore (0.847 ± 0.110), indice di instabilità strutturale su dataset piccoli.

Limiti e sviluppi futuri

Con 60 campioni nel test set (19 deceduti) ogni metrica è soggetta ad alta varianza: un solo paziente classificato diversamente sposta F1 di oltre 0.05. I risultati sono quindi indicativi e richiedono validazione su coorti esterne prima di qualsiasi applicazione clinica. Ulteriori sviluppi potrebbero includere la calibrazione delle probabilità predette e l'integrazione in un sistema di supporto decisionale con supervisione medica — contesto in cui la trasparenza dei modelli rule-based resta il requisito imprescindibile.